fractal

Session 1:

Certification in AI Assisted Coding

A Claude Code Workshop

Arjun and His Transformation

Arjun
Sales Manager

His pain points

Every time he has a good idea, it sits in a document waiting for a developer to look at it.

Building even a rough prototype means filing a ticket and waiting his turn.

His transformation

By the end of this programme, Arjun will be able to build a working prototype by himself and brings it to the next meeting.

No ticket. No queue. No wait.

fractal

Session 1:

Getting Started with Claude Code & AI Tools

Before We Start

Claude Code is pre-installed — no setup today

This is a hands-on session. You will write and run real code, starting in Part 2

When something breaks, that is where learning happens — don't skip the error

Keep a running prompt log as you work — you will use it in Part 3

Questions are welcome throughout — raise your hand or drop them in chat

The learner who asks the most questions today gets a special shout-out in the next session

01 THE AI CODING LANDSCAPE

What’s Changed — And What Hasn’t

What’s Changed

- The speed at which working code can be produced
- How much a non-specialist can build independently
- The cost of a first working draft
- How quickly you can navigate an unfamiliar codebase

What Hasn’t Changed

- Syntax still matters — garbage in, garbage out
- Architecture decisions still require human judgement
- Business logic and edge cases still need you to define them
- Knowing when to stop — the AI is a skill, not a setting

The AI Coding Landscape: Four Platform Tiers

T1 App Builders

FOR

Non-tech founders, designers, PMs

WHAT IT IS

Browser based. Prompt-to-app. UI plus backend plus hosting in one flow.

REPRESENTATIVE TOOLS

Lovable, Bolt.new, v0, Base44, Replit Agent

T2 AI IDEs

FOR

Working developers, technical founders

WHAT IT IS

AI-augmented code editors. You own the code. Multi-file edits, codebase awareness.

REPRESENTATIVE TOOLS

Cursor, Windsurf, GitHub Copilot, Trae

T3 Terminal Agents

FOR

Senior engineers, platform teams

WHAT IT IS

CLI-native agents. Operate on real repos. Highest raw capability on complex codebases.

REPRESENTATIVE TOOLS

Claude Code, Gemini CLI, OpenAI Codex, Devin

T4 Spec-Driven Agents

FOR

Enterprise teams, regulated domains

WHAT IT IS

Spec-first orchestration. Validation gates. Agent execution against versioned specs.

REPRESENTATIVE TOOLS

AWS Kiro, GitHub Spec Kit, OpenSpec, Augment Intent

fractal

02 GETTING STARTED WITH CLAUDE CODE

Claude Code vs. Chat — The Key Difference

Chat Interface (Claude.ai, ChatGPT)

- You paste code in. Claude responds in text. You copy the output back manually.
- Claude has no access to your files.
- Every conversation starts fresh — no memory of your project.
- Best for: **questions, explanations, one-off tasks.**

Claude Code (Terminal)

- Claude runs in your terminal and reads your files directly.
- It proposes changes as diffs — you accept or reject each one.
- It can execute commands on your machine, with your approval.
- Best for: **building, editing, and navigating real projects.**

THE CRITICAL DIFFERENCE: Claude Code acts on your project (new or existing). A chat tool advises on it. One reads your files. The other doesn't know they exist.

That Doesn’t Mean It Has All The Permissions

What Claude Does - Autonomous

READ

Claude reads your files and scans your project structure without asking. This is how it understands context.

You see: file names listed as Claude scans them.

PROPOSE

Claude generates code changes and shows them as diffs. It writes — but does not save — until you decide.

You see: the proposed change before anything is applied.

Where Claude Requires your approval

EDIT

Claude applies changes to your files only after you confirm. Press Y to accept, N to reject.

You control: every write to disk.

EXECUTE

Claude runs terminal commands only after showing you what it intends to run and waiting for your go-ahead.

You control: every command that runs on your machine.

THE PRINCIPLE: Claude proposes. You decide. Nothing changes on your machine without your explicit approval.

Context Is Everything for Coding Assistants

You can run Claude Code. You know the commands. But Claude still knows nothing about your project.

- Claude Code is not a magic box — it works on exactly what you give it
- Every session starts cold — no memory of what you built yesterday, no knowledge of what not to touch, no understanding of how your team works
- Give it a vague brief and it produces reasonable-looking code that solves the wrong problem
- Give it a precise brief and it becomes the fastest developer you have ever worked with

The quality of what you get out is directly proportional to the quality of the context you put in — and that is exactly what CLAUDE.md is built to solve.

What is CLAUDE.md?

CLAUDE.md is a plain text file at your project root. Claude reads it automatically at the start of every session — before you type a single prompt. It is your instruction manual to Claude: what the project is, how it works, and what not to touch.

Project memory

Claude forgets between sessions — CLAUDE.md restores context instantly.

CLAUDE.md — The 5-Section Anatomy

1. Project Overview

What does this project do? Who uses it?

"Patient appointment scheduling system for NHS clinic staff."

2. Tech Stack

Languages, frameworks, DB, libraries. Versions matter.

"Flask 3.0 · SQLAlchemy · SQLite · Python 3.11 · Entry: app.py"

3. Coding Conventions

Naming rules, response format, ORM-only SQL.

"snake_case everywhere. Routes return JSON: {data, error, status}."

4. Do-Not-Touch Zones

Files or logic that must not be modified.

"Do not edit db/schema.sql directly. Do not remove /health route."

5. Useful Context

Anything a new engineer would need to know.

"conflict.py boundary is intentionally strict — do not loosen."

Key Commands

Remember these commands, we will use them shortly.

Commands	What they do	Use when
/init	Scans your project and generates a starter CLAUDE.md automatically.	Starting on a new project for the first time.
/compact	Summarises the conversation so far and continues with a shorter context. Prevents Claude from losing track on long sessions.	You have been working for a while and responses are becoming less accurate.
/usage	Shows how many tokens you have used and the cost of the current session.	You want to monitor spend or sense you are approaching a context limit.
/clear	Starts a completely fresh conversation. Claude forgets everything in the current session.	Responses have drifted and you want a clean slate.

Write your first CLAUDE.md

HANDS-ON LAB

1. **Read the scenario** — account tracker, CSV input, terminal output, no writes, no UI

2. **Brainstorm with plain prompts** — open Claude Code and ask: "What would this script need to do, and what should it never do?" Read what comes back.

3. **Generates a starter CLAUDE.md** from the project context

```
# Shop API

## What this is
An e-commerce backend. It does shopping stuff.
# BAD: "shopping stuff" describes nothing. No user journey, no scope boundary.
#      Claude can't tell what's in scope, so it'll invent features.

## Stack
- Python (recent version)
- A database
- Whatever web framework is cleanest
# BAD: no pinned versions, no named framework, no ORM, no test runner.
#      "Whatever is cleanest" hands an architecture decision to the model
#      and guarantees inconsistency between sessions.

## Conventions
- Write good, clean, modern, production-grade code.
- Follow best practices.
- Handle money correctly.
- Make it fast and scalable.
# BAD: every line is a vibe, not a rule. "Best practices" is unverifiable.
#      "Handle money correctly" — as float? as Decimal? as minor units? Pick one.
#      "Fast and scalable" on a PoC invites premature caching/queues nobody asked for.

## Notes
- Be creative and use your judgment.
- If something is missing, just add it.
- Move fast, we'll clean it up later.
# BAD: this is the opposite of a guardrail. It explicitly licenses scope creep,
#      silent dependency additions, and unreviewed multi-file rewrites.

## Run
- Just run it the normal way.
# BAD: there is no "normal way." No run command, no test command.
#      The one section that should be copy-pasteable is useless.
```

```
# OrderDesk API

## What this is
A small storefront ordering and inventory service. PoC, not production.
One user journey: browse the catalog, place an order, reserve stock, mark it paid or cancelled.

## Stack
- Python 3.12, FastAPI, Uvicorn
- SQLAlchemy ORM on SQLite (file: orderdesk.db) for the PoC
- httpx for the external payment-intent call (mock gateway)
- pydantic v2 for request and response models
- pytest for tests

## Conventions
- Layout: app/main.py, app/db.py, app/models.py, app/schemas.py, app/routes.py
- Type hints on every function. Docstrings on every endpoint.
- Money is stored as integer minor units (paise / cents), never float.
- Prices live on the catalog item. Order lines snapshot the price at order time and never recompute
from the live catalog.
- Stock moves through a reservation, never edited in place. Reserve on order create, release on
cancel, commit on pay.
- Order create is idempotent on an Idempotency-Key header. The same key returns the same order,
never a duplicate.
- Never oversell. Reject the order if reservable stock is below the requested quantity.
- Never hardcode secrets. Read them from environment variables via python-dotenv.

## Run and test
- Run:  python -m uvicorn app.main:app --reload
- Test: python -m pytest -q

## Do not touch
- Do not edit .venv, .git, or orderdesk.db directly.
- Do not add new third-party dependencies without telling me first.
- Do not invent endpoints, statuses, or order states that are not in the brief.
- Do not bypass the reservation flow to adjust stock counts directly.

Always show me a diff and wait for approval before editing more than one file at a time.
```

Thank You.

A strategic partner to the most admired Fortune 500® companies globally, we help power every human decision in the enterprise by bringing advanced analytics & AI, engineering and design.

www.fractal.ai